

RATL Specification

RATL (R-based Array Manipulation Language) is a stack-based, esoteric programming language inspired by [MATL](#). It leverages R's powerful statistical and matrix capabilities through a concise, postfix syntax, making it highly suitable for code golf.

Total symbols: 419 (all 2-byte or shorter)

1. Introduction

RATL is implemented in R and uses R's internal data structures (vectors, lists, matrices) to handle stack operations. It translates RATL code into R code, which is then evaluated.

1.1 The Name

The name stands for **R-based Array Manipulation Language**, reflecting its foundation in R and its primary focus on array-based operations.

1.2 Notation

- **\$n**: Represents the n -th element popped from the stack (starting from the top).
 - **stack**: The global list used for storage.
 - **H, G, L, M**: Clipboards for temporary storage outside the stack.
-

2. The Stack and Data Types

RATL is a stack-based language where every operation interacts with a global stack (an R `list`).

Data Types

RATL uses native R types: - **Numeric**: Integers and floating-point numbers (R `numeric`). - **Logical**: Boolean values (`TRUE`, `FALSE`). - **Character**: Strings of text (R `character`). - **List/Cell Array**: Collections of items, which can be of mixed types (R `list`). - **Matrix**: 2D arrays (R `matrix`).

3. Statements and Separators

- **Space:** Acts as a token separator.
 - **Newline:** Ignored by the parser but acts as a token separator.
 - **Comments:** Lines starting with `#` are treated as comments and ignored.
-

4. Literals

Literals are pushed onto the stack as soon as they are encountered.

4.1 Numbers

Standard numeric notation: 1, 10, 3.14, -5.

4.2 Numerical Arrays (Vectors)

Enclosed in square brackets `[...]`. Elements are separated by spaces. Example: `[1 2 3]` pushes a numeric vector of length 3.

4.3 Character Arrays (Strings)

Enclosed in single quotes `'...'`. Example: `'hello'` pushes a character vector of length 1.

4.4 Cell Arrays (Lists)

Enclosed in curly braces `{...}`. Example: `{1 'a' [1 2]}` pushes an R list containing three different types.

5. Control Flow

RATL uses special symbols for conditional branching and loops.

5.1 Conditional Branching (? ...])

- **Symbols:** ? starts the block,] ends it.
- **If-Else:** ? `truthy` ; `falsy`]. The ; separator distinguishes the “then” and “else” branches.
- **Logic:** Pops the top element. If it’s true, it executes the code until] (or ;). If false and ; is present, it executes the code between ; and].

5.2 While/Repeat Loop (" ... ")

- **Symbol:** "
- **Logic:** Starts a loop. At the end of the loop (marked by another "), it pops the top element. If it’s true, it repeats.

5.3 For-Each Loop ((...))

- **Symbols:** (and)
- **Logic:** (pops an array/list. It then iterates through each element, pushing it onto the stack and executing the code between (and).

5.4 Infinite Loop (` ... `)

- **Symbols:** ` starts and ends the loop.
- **Logic:** Executes the code within the backticks repeatedly. Use the **X** (break) symbol to exit.

5.5 Higher-Order Functions (Block Consumers)

These functions pop a {block} from the stack and apply it to data.

Symbol	Name	Description
q	Map	{block} array q — applies block to each element
e	Filter	{block} array e — keeps elements where block returns truthy
y	Reduce	{block} array y — fold left with block
z	Repeat	N {block} z — executes block N times
@	Execute	{block} @ — pops and executes a block immediately

6. Implicit Actions

6.1 Initial Actions

- The stack is initialized as an empty `list()`.
- A connection to `stdin` is opened for input operations.

6.2 Final Actions

- **Implicit Print:** If the stack contains exactly one element, it is printed automatically. If it contains more than one, the entire stack is printed as a list.
 - **Invisible Output:** The final evaluation result is returned invisibly in R.
-

7. Symbol Reference

7.1 Arithmetic & Comparison

Symbol	R Code	Description	Stack Effect
!	<code>t(\$1)</code>	Transpose	$1 \rightarrow 1$
%	<code>\$2 %% \$1</code>	Modulo	$2 \rightarrow 1$
*	<code>\$2 * \$1</code>	Multiply	$2 \rightarrow 1$
+	<code>\$2 + \$1</code>	Add	$2 \rightarrow 1$
-	<code>\$2 - \$1</code>	Subtract	$2 \rightarrow 1$
/	<code>\$2 / \$1</code>	Divide	$2 \rightarrow 1$
<	<code>\$2 < \$1</code>	Less	$2 \rightarrow 1$
<=	<code>\$2 <= \$1</code>	LessEqual	$2 \rightarrow 1$
=	<code>\$2 == \$1</code>	Equal	$2 \rightarrow 1$
>	<code>\$2 > \$1</code>	Greater	$2 \rightarrow 1$
>=	<code>\$2 >= \$1</code>	GreatEqual	$2 \rightarrow 1$
^	<code>\$2 ^ \$1</code>	Power	$2 \rightarrow 1$
ab	<code>abs(\$1)</code>	Abs	$1 \rightarrow 1$
cl	<code>ceiling(\$1)</code>	Ceiling	$1 \rightarrow 1$
fl	<code>floor(\$1)</code>	Floor	$1 \rightarrow 1$
ro	<code>round(\$1)</code>	Round	$1 \rightarrow 1$
sg	<code>sign(\$1)</code>	Sign	$1 \rightarrow 1$

Symbol	R Code	Description	Stack Effect
<code>sq</code>	<code>sqrt(\$1)</code>	Sqrt	$1 \rightarrow 1$
<code>tr</code>	<code>trunc(\$1)</code>	Trunc	$1 \rightarrow 1$
<code>~</code>	<code>!\$1</code>	Not	$1 \rightarrow 1$

7.2 Stack & Control

Symbol	R Code	Description	Stack Effect
<code>D</code>	<code>list(\$1, \$1)</code>	Duplicate	$1 \rightarrow 2$
<code>Ls</code>	<code>length(stack)</code>	Stack Length	$0 \rightarrow 1$
<code>U</code>	<code>for(x in \$1)</code> <code>stack[[length(stack)+1]]</code> <code><- x</code>	Unpack	$1 \rightarrow 0$
<code>i</code>	<code>scan(ratl_stdin, Input</code> <code>what=character(),</code> <code>n=1,</code> <code>quiet=TRUE)</code>	Input	$0 \rightarrow 1$
<code>w</code>	<code>list(\$1, \$2)</code>	Swap	$2 \rightarrow 2$
<code>x</code>	<code>NULL</code>	Delete	$1 \rightarrow 0$

7.3 Higher-Order Functions

Symbol	R Code	Description	Stack Effect
<code>Fq</code>	<code>NULL</code>	Map (evaluator)	$0 \rightarrow 1$
<code>Fr</code>	<code>Reduce(\$2, \$1)</code>	Reduce	$2 \rightarrow 1$
<code>Ft</code>	<code>NULL</code>	Filter (evaluator)	$0 \rightarrow 1$
<code>Fx</code>	<code>NULL</code>	Repeat (evaluator)	$0 \rightarrow 1$
<code>e</code>	<code>NULL</code>	Filter (evaluator)	$0 \rightarrow 1$
<code>q</code>	<code>NULL</code>	Map (evaluator)	$0 \rightarrow 1$
<code>y</code>	<code>NULL</code>	Reduce (evaluator)	$0 \rightarrow 1$
<code>z</code>	<code>NULL</code>	Repeat (evaluator)	$0 \rightarrow 1$

7.4 Array Operations

Symbol	R Code	Description	Stack Effect
:	<code>seq(\$2, \$1)</code>	Sequence	$2 \rightarrow 1$
O	<code>sort(\$1)</code>	Sort	$1 \rightarrow 1$
R	<code>rev(\$1)</code>	Reverse	$1 \rightarrow 1$
SH	<code>head(\$1)</code>	Head	$1 \rightarrow 1$
SR	<code>rank(\$1)</code>	Rank	$1 \rightarrow 1$
ST	<code>tail(\$1)</code>	Tail	$1 \rightarrow 1$
el	<code>\$2[[\$1]]</code>	Extract Element [[	$2 \rightarrow 1$
en	<code>\$2[[as.character(\$1)]]</code>	Extract Name \$	$2 \rightarrow 1$
es	<code>\$2[\$1]</code>	Extract Subset [	$2 \rightarrow 1$
fE	<code>head(\$1, 1)</code>	FirstElem	$1 \rightarrow 1$
fu	<code>unlist(\$1)</code>	Flatten	$1 \rightarrow 1$
hd	<code>head(\$2, \$1)</code>	HeadN	$2 \rightarrow 1$
ix	<code>match(\$2, \$1)</code>	IndexOf	$2 \rightarrow 1$
l	<code>length(\$1)</code>	Length	$1 \rightarrow 1$
la	<code>tail(\$1, 1)</code>	LastElem	$1 \rightarrow 1$
rv	<code>paste(rev(strsplit(as.character(\$1), "")[[1]]), collapse="")</code>	Reverse String	$1 \rightarrow 1$
tb	<code>tabulate(\$1)</code>	Tabulate	$1 \rightarrow 1$
tl	<code>tail(\$2, \$1)</code>	TailN	$2 \rightarrow 1$
u	<code>unique(\$1)</code>	Unique	$1 \rightarrow 1$
un	<code>length(unique(\$1))</code>	UniqueN	$1 \rightarrow 1$
vH	<code>which(\$1, arr.ind=TRUE)</code>	WhichArr	$1 \rightarrow 1$
vW	<code>seq(\$3, \$2, length.out=\$1)</code>	SeqLen	$3 \rightarrow 1$
wh	<code>which(\$1)</code>	Which	$1 \rightarrow 1$
zp	<code>unlist(Map(list Zip \$2, \$1))</code>		$2 \rightarrow 1$

7.5 Matrix

Symbol	R Code	Description	Stack Effect
Bi	<code>IQR(\$1)</code>	IQR	$1 \rightarrow 1$
R9	<code>apply(t(\$1), 2, rev)</code>	Rot90	$1 \rightarrow 1$
Y!	<code>matrix(\$3, nrow=\$2, ncol=\$1)</code>	Create Matrix	$3 \rightarrow 1$

Symbol	R Code	Description	Stack Effect
Y*	\$2 %*% \$1	Matrix Mult	2 → 1
YM	matrix(\$3, nrow=\$2, ncol=\$1, byrow=TRUE)	Matrix ByRow	3 → 1
mM	model.matrix(\$1)	Model Matrix	1 → 1
v2	solve(\$2, \$1)	Solve 2	2 → 1
vL	solve(\$1)	Solve	1 → 1
vQ	qr(\$1)	QR Decomp	1 → 1
vV	svd(\$1)	SVD	1 → 1
y!	diag(\$1)	Diag	1 → 1
yD	diag(\$1)	Identity Matrix	1 → 1
yc	eigen(\$1)\$vectors	EigenVectors	1 → 1
yd	det(\$1)	Determinant	1 → 1
yf	as.matrix(\$1)	Full	1 → 1
yk	kronecker(\$2, \$1)	Kronecker	2 → 1
yl	{x<-\$1; x[upper.tri(x)]<- 0; x}	TriLower	1 → 1
ym	matrix(\$3, nrow=\$2, ncol=\$1)	Create Matrix	3 → 1
yt	sum(diag(\$1))	Trace	1 → 1
yu	{x<-\$1; x[lower.tri(x)]<- 0; x}	TriUpper	1 → 1
yv	eigen(\$1)\$values	EigenValues	1 → 1

7.6 Statistics

Symbol	R Code	Description	Stack Effect
B5	fivenum(\$1)	FiveNum	1 → 1
BS	summary(\$1)	Summary	1 → 1
Ba	mad(\$1)	MAD	1 → 1
Bn	min(\$1)	Min	1 → 1
Br	range(\$1)	Range	1 → 1
Bx	max(\$1)	Max	1 → 1

Symbol	R Code	Description	Stack Effect
Ku	<code>sum(($\\$1 - \text{mean}(\\$1)$)⁴)/(($\text{length}(\\$1) - 1$) * $\text{sd}(\\$1)$⁴)</code>	Kurtosis	1 → 1
Sd	<code>ratl_cum_sd($\\$1$)</code>	CumSD	1 → 1
Sk	<code>sum(($\\$1 - \text{mean}(\\$1)$)³)/(($\text{length}(\\$1) - 1$) * $\text{sd}(\\$1)$³)</code>	Skewness	1 → 1
Sm	<code>mean($\\$1$, na.rm=TRUE)</code>	MeanNA	1 → 1
Sn	<code>sum($\\$1$, na.rm=TRUE)</code>	SumNA	1 → 1
Sw	<code>weighted.mean($\\$2$, $\\$1$)</code>	WeightedMean	2 → 1
V	<code>var($\\$1$)</code>	Variance	1 → 1
h	<code>median($\\$1$)</code>	Median	1 → 1
m	<code>mean($\\$1$)</code>	Mean	1 → 1
mn	<code>min($\\$2$, $\\$1$)</code>	Min2	2 → 1
mx	<code>max($\\$2$, $\\$1$)</code>	Max2	2 → 1
r1	<code>1:$\\$1$</code>	Range1toN	1 → 1
s	<code>sum($\\$1$)</code>	Sum	1 → 1
sm	<code>summary($\\$1$)</code>	Summary	1 → 1
v	<code>var($\\$1$)</code>	Variance	1 → 1
vB	<code>pmin($\\$2$, $\\$1$)</code>	PMin	2 → 1
vE	<code>rowSums($\\$1$)</code>	Row Sums	1 → 1
vG	<code>pmax($\\$2$, $\\$1$)</code>	PMax	2 → 1
vM	<code>colMeans($\\$1$)</code>	Col Means	1 → 1
vN	<code>rowMeans($\\$1$)</code>	Row Means	1 → 1
vS	<code>colSums($\\$1$)</code>	Col Sums	1 → 1

7.7 Statistical Modeling

Symbol	R Code	Description	Stack Effect
DN	<code>dnorm($\\$3$, $\\$2$, $\\$1$)</code>	Density Normal	3 → 1
K!	<code>coef($\\$1$)</code>	Coef	1 → 1
KB	<code>BIC($\\$1$)</code>	BIC	1 → 1
KC	<code>cor.test($\\$2$, $\\$1$)</code>	Cor Test	2 → 1
KE	<code>residuals($\\$1$)</code>	Residuals	1 → 1
KF	<code>fitted($\\$1$)</code>	Fitted	1 → 1
KL	<code>logLik($\\$1$)</code>	LogLik	1 → 1

Symbol	R Code	Description	Stack Effect
KV	vcov(\$1)	VCov	1 → 1
av	anova(\$1)	Anova	1 → 1
dB	dbinom(\$3, \$2, \$1)	Density Binomial	3 → 1
dC	dchisq(\$2, \$1)	Density Chi-Square	2 → 1
dG	dgeom(\$2, \$1)	Density Geometric	2 → 1
dL	dlogis(\$3, \$2, \$1)	Density Logistic	3 → 1
dN	dnbinom(\$3, \$2, \$1)	Density Neg-Binomial	3 → 1
dP	dpois(\$2, \$1)	Density Poisson	2 → 1
db	dbeta(\$3, \$2, \$1)	Density Beta	3 → 1
dc	dcauchy(\$3, \$2, \$1)	Density Cauchy	3 → 1
de	dexp(\$2, \$1)	Density Exponential	2 → 1
df	df(\$3, \$2, \$1)	Density F	3 → 1
dg	dgamma(\$3, \$2, \$1)	Density Gamma	3 → 1
dh	dhyper(\$4, \$3, \$2, \$1)	Density Hypergeometric	4 → 1
dI	dlnorm(\$3, \$2, \$1)	Density Log-Normal	3 → 1
dn	dnorm(\$1)	Density Normal	1 → 1
dt	dt(\$2, \$1)	Density Student-t	2 → 1
du	dunif(\$3, \$2, \$1)	Density Uniform	3 → 1
dw	dweibull(\$3, \$2, \$1)	Density Weibull	3 → 1
ka	anova(\$1)	Anova	1 → 1
kb	bartlett.test(\$1)	Bartlett Test	1 → 1
kc	chisq.test(\$1)	Chi-Square Test	1 → 1
ke	predict(\$2, \$1)	Predict	2 → 1
kf	fisher.test(\$1)	Fisher Test	1 → 1
kg	glm(\$1)	GLM	1 → 1
ki	AIC(\$1)	AIC	1 → 1
kk	ks.test(\$2, \$1)	KS Test	2 → 1
kl	lm(\$1)	Linear Model	1 → 1
kn	nls(\$1)	NLS	1 → 1
ko	loess(\$1)	Loess	1 → 1
kp	prop.test(\$2, \$1)	Prop Test	2 → 1
kt	t.test(\$2, \$1)	T-Test	2 → 1
kv	var.test(\$2, \$1)	F-Test Var	2 → 1
kW	wilcox.test(\$2, \$1)	Wilcoxon Test	2 → 1
lm	lm(\$1)	Linear Model	1 → 1
mC	confint(\$1)	Conf Int	1 → 1
mL	lm(\$2 ~ \$1)	LM Simple	2 → 1
mO	offset(\$1)	Offset	1 → 1
mP	predict(\$1)	Predict	1 → 1
mR	model.frame(\$1)	Model Frame	1 → 1
mT	terms(\$1)	Terms	1 → 1

Symbol	R Code	Description	Stack Effect
mU	update(\$2, \$1)	Update	2 → 1
mW	formula(\$1)	Formula	1 → 1
tt	t.test(\$2, \$1)	T-Test	2 → 1
vd	density(\$1)	Density	1 → 1
vp	prcomp(\$1)	PCA	1 → 1

7.8 Distributions & Tests

Symbol	R Code	Description	Stack Effect
BC	cov(\$2, \$1)	Covariance	2 → 1
Bc	cor(\$2, \$1)	Correlation	2 → 1
KK	kruskal.test(\$1)	Kruskal-Wallis	1 → 1
LG	lgamma(\$1)	Log Gamma	1 → 1
N	rnorm(\$1)	Random Normal	1 → 1
PN	pnorm(\$3, \$2, \$1)	Prob Normal	3 → 1
QN	qnorm(\$3, \$2, \$1)	Quantile Normal	3 → 1
RB	rbeta(\$3, \$2, \$1)	Random Beta	3 → 1
RE	rexp(\$2, \$1)	Random Exponential	2 → 1
RN	rnorm(\$3, \$2, \$1)	Random Normal	3 → 1
RU	runif(\$3, \$2, \$1)	Random Uniform	3 → 1
cr	cor(\$2, \$1)	Correlation	2 → 1
cv	cov(\$2, \$1)	Covariance	2 → 1
di	digamma(\$1)	Digamma	1 → 1
e-	expm1(\$1)	Exp(x)-1	1 → 1
ex	exp(\$1)	Exp	1 → 1
ks	shapiro.test(\$1)	Shapiro-Wilk	1 → 1
l+	log1p(\$1)	Log(1+x)	1 → 1
l1	log10(\$1)	Log 10	1 → 1
l2	log2(\$1)	Log 2	1 → 1
lb	lbeta(\$2, \$1)	Log Beta	2 → 1
lg	log(\$1)	Log Natural	1 → 1
mB	beta(\$2, \$1)	Beta	2 → 1
mG	gamma(\$1)	Gamma	1 → 1
pB	pbinom(\$3, \$2, \$1)	Prob Binomial	3 → 1
pG	pgeom(\$2, \$1)	Prob Geometric	2 → 1
pL	plogis(\$3, \$2, \$1)	Prob Logistic	3 → 1
pN	pnbinom(\$3, \$2, \$1)	Prob Neg-Binomial	3 → 1
pP	ppois(\$2, \$1)	Prob Poisson	2 → 1
pb	pbeta(\$3, \$2, \$1)	Prob Beta	3 → 1

Symbol	R Code	Description	Stack Effect
pc	pcauchy(\$3, \$2, \$1)	Prob Cauchy	3 → 1
pe	pexp(\$2, \$1)	Prob Exponential	2 → 1
pg	pgamma(\$3, \$2, \$1)	Prob Gamma	3 → 1
ph	phyper(\$4, \$3, \$2, \$1)	Prob Hypergeometric	4 → 1
pl	plnorm(\$3, \$2, \$1)	Prob Log-Normal	3 → 1
pn	pnorm(\$1)	Prob Normal	1 → 1
ps	psigamma(\$2, \$1)	Psigamma	2 → 1
pt	pt(\$2, \$1)	Prob Student-t	2 → 1
pu	punif(\$3, \$2, \$1)	Prob Uniform	3 → 1
pw	pweibull(\$3, \$2, \$1)	Prob Weibull	3 → 1
qB	qbinom(\$3, \$2, \$1)	Quantile Binomial	3 → 1
qG	qgeom(\$2, \$1)	Quantile Geometric	2 → 1
qL	qlogis(\$3, \$2, \$1)	Quantile Logistic	3 → 1
qN	qnbinom(\$3, \$2, \$1)	Quantile Neg-Binomial	3 → 1
qP	qpois(\$2, \$1)	Quantile Poisson	2 → 1
qb	qbeta(\$3, \$2, \$1)	Quantile Beta	3 → 1
qc	qcauchy(\$3, \$2, \$1)	Quantile Cauchy	3 → 1
qe	qexp(\$2, \$1)	Quantile Exponential	2 → 1
qg	qgamma(\$3, \$2, \$1)	Quantile Gamma	3 → 1
qh	qhyper(\$4, \$3, \$2, \$1)	Quantile Hypergeometric	4 → 1
ql	qlnorm(\$3, \$2, \$1)	Quantile Log-Normal	3 → 1
qn	qnorm(\$1)	Quantile Normal	1 → 1
qt	qt(\$2, \$1)	Quantile Student-t	2 → 1
qu	qunif(\$3, \$2, \$1)	Quantile Uniform	3 → 1
qw	qweibull(\$3, \$2, \$1)	Quantile Weibull	3 → 1
rB	rbinom(\$3, \$2, \$1)	Random Binomial	3 → 1
rG	rgeom(\$2, \$1)	Random Geometric	2 → 1
rL	rlogis(\$3, \$2, \$1)	Random Logistic	3 → 1
rN	rnbinom(\$3, \$2, \$1)	Random Neg-Binomial	3 → 1
rP	rpois(\$2, \$1)	Random Poisson	2 → 1
rb	rbinom(\$1, size=1, prob=0.5)	Random Binomial	1 → 1
rc	rcauchy(\$3, \$2, \$1)	Random Cauchy	3 → 1
re	rexp(\$1)	Random Exp	1 → 1
rg	rgamma(\$3, \$2, \$1)	Random Gamma	3 → 1
rh	rhyper(\$4, \$3, \$2, \$1)	Random Hypergeometric	4 → 1
rl	rlnorm(\$3, \$2, \$1)	Random Log-Normal	3 → 1
rp	rpois(\$1, lambda=1)	Random Poisson	1 → 1
rt	rt(\$2, \$1)	Random Student-t	2 → 1
rw	rweibull(\$3, \$2, \$1)	Random Weibull	3 → 1
s4	logical(0)	EmptyLog	0 → 1
sR	regexpr(\$2, \$1)	Regexpr	2 → 1

Symbol	R Code	Description	Stack Effect
sX	gregexpr(\$2, \$1)	Gregexpr	2 → 1
tg	trigamma(\$1)	Trigamma	1 → 1

7.9 Math Functions

Symbol	R Code	Description	Stack Effect
Bs	sd(\$1)	Standard Deviation	1 → 1
SP	sinpi(\$1)	Sin(pi*x)	1 → 1
XC	cumsum(\$1)	CumSum	1 → 1
a2	atan2(\$2, \$1)	ArcTan2	2 → 1
aC	acos(\$1)	ArcCos	1 → 1
aS	asin(\$1)	ArcSin	1 → 1
aT	atan(\$1)	ArcTan	1 → 1
ac	acosh(\$1)	ArcCosh	1 → 1
as	asinh(\$1)	ArcSinh	1 → 1
at	atanh(\$1)	ArcTanh	1 → 1
bA	bitwAnd(\$2, \$1)	BitAnd	2 → 1
c1	cumprod(\$1)	CumProd	1 → 1
ch	cosh(\$1)	Cosh	1 → 1
cn	is.element(\$2, \$1)	IsIn	2 → 1
cp	cospi(\$1)	Cos(pi*x)	1 → 1
mc	cos(\$1)	Cos	1 → 1
ms	sin(\$1)	Sin	1 → 1
mt	tan(\$1)	Tan	1 → 1
sD	setdiff(\$2, \$1)	SetDiff	2 → 1
sd	sd(\$1)	Standard Deviation	1 → 1
sh	sinh(\$1)	Sinh	1 → 1
th	tanh(\$1)	Tanh	1 → 1
tp	tanpi(\$1)	Tan(pi*x)	1 → 1
vF	diff(\$1)	Diff	1 → 1
vI	cummin(\$1)	CumMin	1 → 1
vX	cummax(\$1)	CumMax	1 → 1

7.10 Combinatorics & Special

Symbol	R Code	Description	Stack Effect
MC	choose(\$2, \$1)	Choose	2 → 1

Symbol	R Code	Description	Stack Effect
Xn	choose(\$2, \$1)	Choose (nCr)	$2 \rightarrow 1$
fp	factorial(\$1)	Factorial	$1 \rightarrow 1$

7.11 Complex Numbers

Symbol	R Code	Description	Stack Effect
cA	Arg(\$1)	Arg	$1 \rightarrow 1$
cI	Im(\$1)	Imag Part	$1 \rightarrow 1$
cJ	Conj(\$1)	Conjugate	$1 \rightarrow 1$
cM	Mod(\$1)	Modulus	$1 \rightarrow 1$
cR	Re(\$1)	Real Part	$1 \rightarrow 1$

7.12 String Operations

Symbol	R Code	Description	Stack Effect
,	c(\$2, \$1)	Concat Vectors	$2 \rightarrow 1$
C	paste0(\$2, \$1)	Concat	$2 \rightarrow 1$
J	paste(\$2, collapse=\$1)	Join List	$2 \rightarrow 1$
S	strsplit(\$2, \$1)[[1]]	Split String	$2 \rightarrow 1$
S!	chartr(\$2, \$1, \$3)	Translate	$3 \rightarrow 1$
SG	gsub(\$2, \$1, \$3)	GSub	$3 \rightarrow 1$
Xc	chartr(\$1, \$2, \$3)	Char Translate	$3 \rightarrow 1$
Xt	tolower(\$1)	ToLower	$1 \rightarrow 1$
a	as.character(\$1)	To String	$1 \rightarrow 1$
c	intToUtf8(\$1)	To Char	$1 \rightarrow 1$
dS	split(\$2, \$1)	Split	$2 \rightarrow 1$
dU	unsplit(\$2, \$1)	Unsplit	$2 \rightarrow 1$
j	paste(\$2, \$1)	Join	$2 \rightarrow 1$
sB	grepl(\$2, \$1)	Grepl	$2 \rightarrow 1$
sG	grep(\$2, \$1)	Grep	$2 \rightarrow 1$
sL	tolower(\$1)	ToLower	$1 \rightarrow 1$
sM	message(\$1)	Message	$1 \rightarrow 0$
sQ	character(0)	EmptyChar	$0 \rightarrow 1$
sS	stop(\$1)	Stop	$1 \rightarrow 0$
sU	toupper(\$1)	ToUpper	$1 \rightarrow 1$
sW	warning(\$1)	Warning	$1 \rightarrow 0$
sf	sprintf(\$2, \$1)	Sprintf	$2 \rightarrow 1$

Symbol	R Code	Description	Stack Effect
sn	nchar(\$1)	NChar	1 → 1
ss	substr(\$3, \$2, \$1)	Substr	3 → 1
st	trimws(\$1)	TrimWS	1 → 1

7.13 Set Operations

Symbol	R Code	Description	Stack Effect
sF	options(prompt=\$1)	SetPrompt	1 → 0
sI	intersect(\$2, \$1)	Intersect	2 → 1
sJ	setwd(\$1)	SetWD	1 → 0
sN	union(\$2, \$1)	Union	2 → 1
sY	options(scipen=\$1)	SetScipen	1 → 0
se	Sys.setenv(...)	Setenv	1 → 0

7.14 Bitwise Operations

Symbol	R Code	Description	Stack Effect
bL	bitwShiftL(\$2, \$1)	BitShiftL	2 → 1
bN	bitwNot(\$1)	BitNot	1 → 1
bO	bitwOr(\$2, \$1)	BitOr	2 → 1
bR	bitwShiftR(\$2, \$1)	BitShiftR	2 → 1
bX	bitwXor(\$2, \$1)	BitXor	2 → 1

7.15 Type & Introspection

Symbol	R Code	Description	Stack Effect
AS	as(\$2, \$1)	As Type	2 → 1
Xq	ratl_is_prime(\$1)	Is Prime	1 → 1
is	is(\$2, \$1)	Is Type	2 → 1
mp	ratl_is_prime(\$1)	Is Prime	1 → 1
oA	attr(\$2, \$1)	Attr	2 → 1
oC	colnames(\$1)	Colnames	1 → 1
oa	attributes(\$1)	Attributes	1 → 1
oc	class(\$1)	Class	1 → 1
od	dim(\$1)	Dim	1 → 1

Symbol	R Code	Description	Stack Effect
on	names(\$1)	Names	1 → 1
or	rownames(\$1)	Rownames	1 → 1
ot	typeof(\$1)	Typeof	1 → 1
ou	unlist(\$1)	Unlist	1 → 1

7.16 File I/O

Symbol	R Code	Description	Stack Effect
F	readLines(\$1)	Read File	1 → 1
FL	list.files(\$1)	List Files	1 → 1
FW	{writeLines(as.character(\$2), \$1); NULL}	Write File	2 → 0
W	writeLines(as.character(\$2), \$1); NULL	Write File	2 → 0
fA	dirname(\$1)	Dirname	1 → 1
fC	write.csv(\$2, \$1)	Write CSV	2 → 0
fD	list.dirs(\$1)	List Dirs	1 → 1
fN	file.create(\$1)	File Create	1 → 1
fP	normalizePath(\$1)	abs Path	1 → 1
fT	tempfile()	Temp File	0 → 1
fX	dir.exists(\$1)	Dir Exists	1 → 1
fb	basename(\$1)	Basename	1 → 1
fc	read.csv(\$1)	Read CSV	1 → 1
fd	dir.create(\$1)	Dir Create	1 → 1
fe	file.exists(\$1)	File Exists	1 → 1
fi	file.info(\$1)	File Info	1 → 1
fm	file.remove(\$1)	File Remove	1 → 1
fr	read.table(\$1)	Read Table	1 → 1
ft	tempdir()	Temp Dir	0 → 1
fw	write.table(\$2, \$1)	Write Table	2 → 0

7.17 System

Symbol	R Code	Description	Stack Effect
T	<code>as.numeric(Sys.time())</code>	Time Epoch	$0 \rightarrow 1$
ZT	<code>Sys.timezone()</code>	Timezone	$0 \rightarrow 1$
ge	<code>Sys.getenv(\$1)</code>	Getenv	$1 \rightarrow 1$
pi	<code>Sys.getpid()</code>	GetPID	$0 \rightarrow 1$
sK	<code>proc.time() - \$1</code>	Toc	$1 \rightarrow 1$
sP	<code>Sys.sleep(\$1)</code>	Sleep	$1 \rightarrow 0$
sT	<code>proc.time()</code>	Tic	$0 \rightarrow 1$
sV	<code>R.version.string</code>	RVersion	$0 \rightarrow 1$
um	<code>Sys.umask(\$1)</code>	Umask	$1 \rightarrow 1$
zg	<code>gc()</code>	GC	$0 \rightarrow 1$
zl	<code>Sys.localeconv()</code>	Locale	$0 \rightarrow 1$
zo	<code>options()</code>	Options	$0 \rightarrow 1$
zt	<code>Sys.time()</code>	SysTime	$0 \rightarrow 1$
zv	<code>version</code>	Version	$0 \rightarrow 1$

7.18 Other

Symbol	R Code	Description	Stack Effect
#	<code>\$2[\$1]</code>	Filter	$2 \rightarrow 1$
&	<code>\$2 %o% \$1</code>	Outer Product	$2 \rightarrow 1$
..	<code>list(\$2, \$1)</code>	Pair	$2 \rightarrow 1$
A	<code>utf8ToInt(\$1)</code>	To ASCII	$1 \rightarrow 1$
BT	<code>{idx<-c(1,4,3,2,5); \$1[idx,idx]}</code>	Bingo Twin	$1 \rightarrow 1$
Bz	<code>scale(\$1)</code>	Scale	$1 \rightarrow 1$
Fa	<code>apply(\$3, \$2, \$1)</code>	Apply	$3 \rightarrow 1$
Ff	<code>Filter(\$2, \$1)</code>	Filter (Func)	$2 \rightarrow 1$
Fl	<code>lapply(\$2, \$1)</code>	Lapply	$2 \rightarrow 1$
Fm	<code>mapply(\$1, ...)</code>	Mapply	$1 \rightarrow 1$
Fn	<code>Find(\$2, \$1)</code>	Find (Func)	$2 \rightarrow 1$
Fp	<code>Position(\$2, \$1)</code>	Position	$2 \rightarrow 1$
Fs	<code>sapply(\$2, \$1)</code>	Sapply	$2 \rightarrow 1$
Fv	<code>vapply(\$3, \$2, \$1)</code>	Vapply	$3 \rightarrow 1$
In	<code>Inf</code>	Inf	$0 \rightarrow 1$
KA	<code>aov(\$1)</code>	AOV	$1 \rightarrow 1$
Na	<code>NA</code>	NA	$0 \rightarrow 1$
P	<code>prod(\$1)</code>	Product	$1 \rightarrow 1$
Pi	<code>pi</code>	Pi	$0 \rightarrow 1$
Q	<code>quantile(\$1)</code>	Quantile	$1 \rightarrow 1$
XP	<code>ratl_prime_factors(\$1)</code>	Prime Factors	$1 \rightarrow 1$

Symbol	R Code	Description	Stack Effect
Xb	ratl_bin(\$1)	To Binary	1 → 1
Xd	as.Date(\$1)	To Date	1 → 1
Xk	{r <- rle(\$1); list(r\$values, r\$lengths)}	RLE	1 → 2
Xo	ratl_mode(\$1)	Mode	1 → 1
Xr	ratl_rms(\$1)	RMS	1 → 1
Xs	sd(\$1)	Std Dev	1 → 1
dA	aggregate(\$3, \$2, \$1)	Aggregate	3 → 1
dE	cut(\$2, \$1)	Cut	2 → 1
dO	order(\$1)	Order	1 → 1
dT	table(\$1)	Table	1 → 1
dX	xtabs(\$2, \$1)	XTabs	2 → 1
dY	by(\$3, \$2, \$1)	By	3 → 1
fa	ratl_factors(\$1)	Factors	1 → 1
fn	Negate(\$1)	Negate	1 → 1
g	ratl_gcd(\$2, \$1)	GCD	2 → 1
lA	any(\$1)	Any	1 → 1
lX	xor(\$2, \$1)	Xor	2 → 1
lZ	sum(\$1 != 0)	NNZ	1 → 1
lc	ratl_lcm(\$2, \$1)	LCM	2 → 1
mF	factor(\$1)	Factor	1 → 1
mI	\$2 %/% \$1	IntDiv	2 → 1
mQ	quantile(\$1, \$2)	Quantile	2 → 1
mf	ratl_prime_factors(\$1)	Prime Factors	1 → 1
ml	ratl_lcm(\$2, \$1)	LCM	2 → 1
n	as.numeric(\$1)	To Number	1 → 1
ns	\$2[length(\$2)+1-\$1]	NegSlice	2 → 1
p	ratl_print(\$1)	Print	1 → 0
pC	pchisq(\$2, \$1)	Prob Chi-Square	2 → 1
pf	pf(\$3, \$2, \$1)	Prob F	3 → 1
qC	qchisq(\$2, \$1)	Quantile Chi-Square	2 → 1
qf	qf(\$3, \$2, \$1)	Quantile F	3 → 1
r	runif(\$1)	Random (n)	1 → 1
r2	rep(\$2, \$1)	Rep	2 → 1
rC	rchisq(\$2, \$1)	Random Chi-Square	2 → 1
rf	rf(\$3, \$2, \$1)	Random F	3 → 1
rm	ratl_rms(\$1)	RMS	1 → 1
sC	cat(\$1)	Cat	1 → 0
sE	date()	Date	0 → 1
sH	getwd()	GetWD	0 → 1
sZ	numeric(0)	EmptyNum	0 → 1
sc	scale(\$1)	Scale	1 → 1

Symbol	R Code	Description	Stack Effect
sr	sub(\$2, \$1, \$3)	Sub	3 $\rightarrow$ 1
vA	all(\$1)	All	1 $\rightarrow$ 1
vC	col(\$1)	Col Indices	1 $\rightarrow$ 1
vD	drop(\$1)	Drop	1 $\rightarrow$ 1
vR	row(\$1)	Row Indices	1 $\rightarrow$ 1
vT	matrix(rep(\$3, \$2*\$1), nrow=nrow(\$3)*\$2)	RepMat	3 $\rightarrow$ 1
zd	as.Date(\$1)	To Date	1 $\rightarrow$ 1
\	\$2 \ \$1	Or	2 $\rightarrow$ 1